

Data Exploration

Data Set Overview

The table below lists each of the files available for analysis with a short description of what is found in each one.

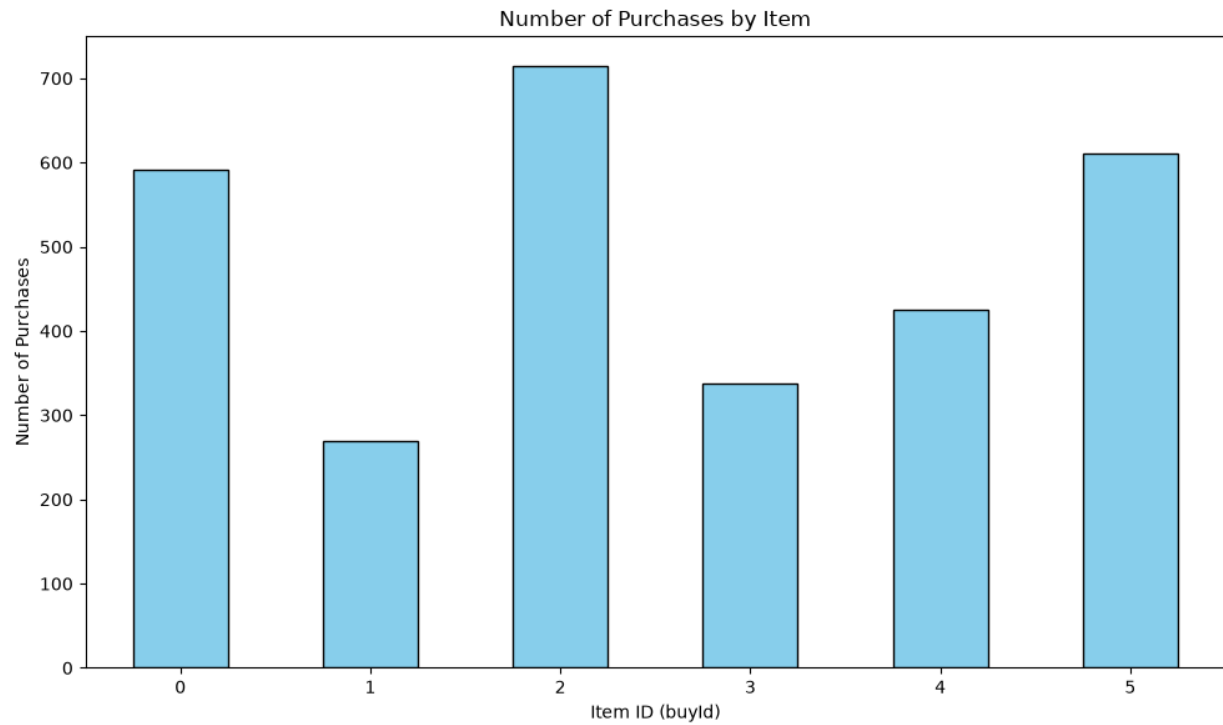
File Name	Description	Fields
ad-clicks.csv	Represents individual ad clicks made by users within the game.	timestamp: Time of click txId: Transaction ID userSessionId: Session identifier teamId: User's team ID userId: User identifier adId: Advertisement identifier adCategory: Category of the ad
buy-clicks.csv	Records in-game microtransaction purchases made by users.	timestamp: Time of purchase txId: Transaction ID userSessionId: Session identifier team: User's team ID userId: User identifier buyId: Item identifier purchased price: Cost of the item
game-clicks.csv	Logs the gameplay clicks made by users and whether they hit the target.	timestamp: Time of click clickId: Unique click identifier userId: User identifier userSessionId: Session identifier isHit: Boolean/integer indicating if the click was a hit (1) or miss (0) teamId: User's team ID teamLevel: Team's current level
level-events.csv	Tracks events related to team level progression (start and end).	timestamp: Time of event eventId: Event identifier teamId: Team identifier teamLevel: Level being played eventType: Start or end of level
team-assignments.csv	Maps users to the teams they are assigned to.	timestamp: Time of assignment team: Team identifier userId: User identifier assignmentId: Assignment identifier
team.csv	Contains metadata and state information for each team.	teamId: Team identifier name: Team name teamCreationTime: Timestamp of creation teamEndTime: Timestamp when

		team ended strength: Team strength metric currentLevel: Team's current progression level
user-session.csv	Logs the start and end of user gameplay sessions and their platform.	timestamp: Time of session event userSessionId: Session identifier userId: User identifier teamId: User's team ID assignmentId: Assignment identifier sessionType: Event type (start/end) teamLevel: Team's level during session platformType: OS/Platform used
users.csv	Contains demographic and profile metadata for registered users.	timestamp: Time of registration userId: User identifier nick: User's nickname twitter: User's Twitter handle dob: Date of birth country: Country of residence

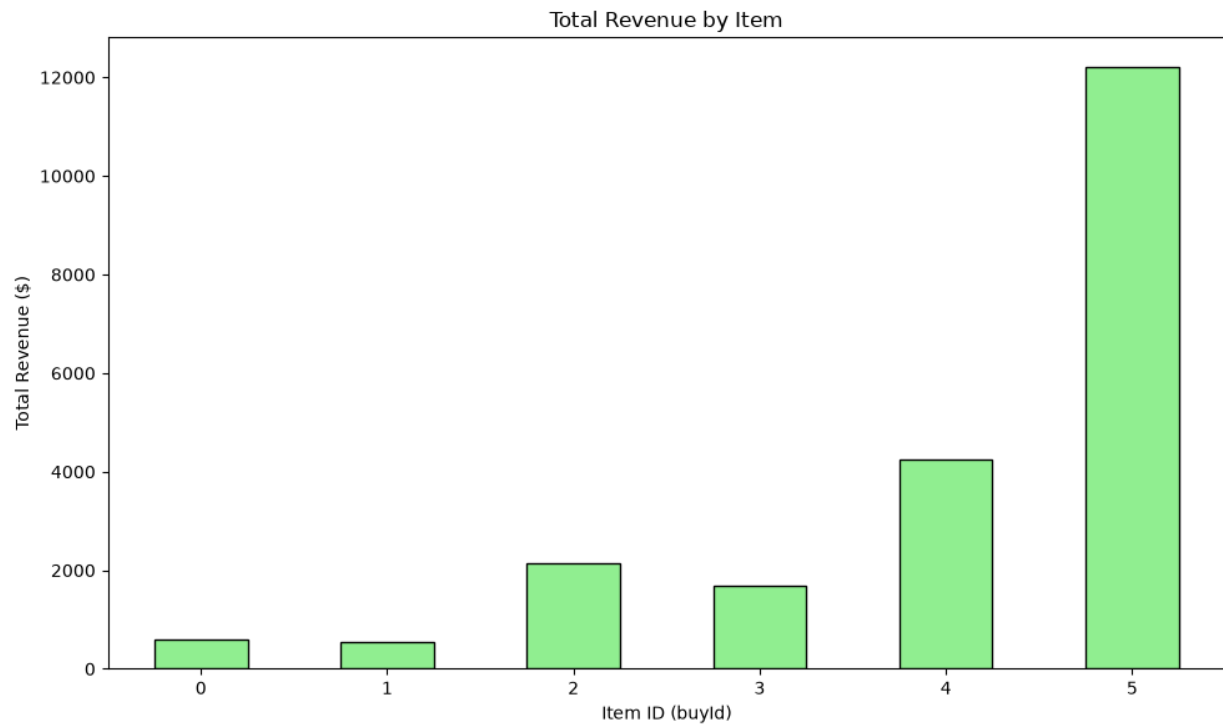
Aggregation

Amount spent buying items	\$21,407.00
Number of unique items available to be purchased	6

A histogram showing how many times each item is purchased:

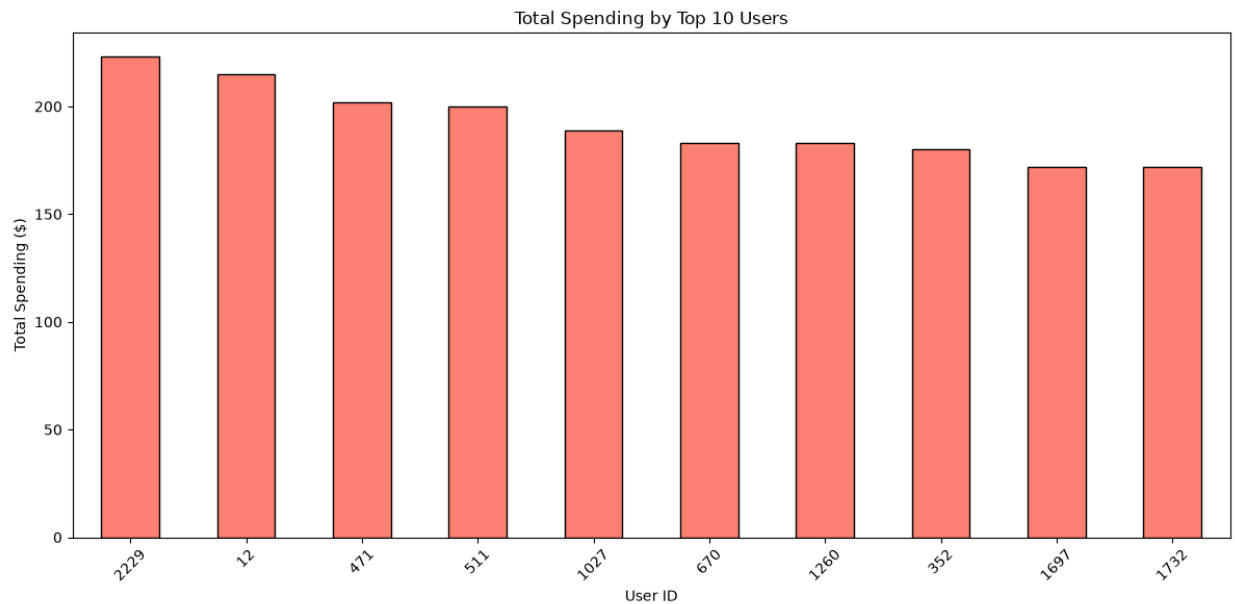


A histogram showing how much money was made from each item:



Filtering

A histogram showing total amount of money spent by the top ten users (ranked by how much money they spent).



The following table shows the user id, platform, and hit-ratio percentage for the top three buying users:

Rank	User Id	Platform	Hit-Ratio (%)
1	2229	iphone	11.60
2	12	iphone	13.07
3	471	iphone	14.50

Analysis Conclusions

The final KNIME workflow is shown below:

KNIME screenshot to be inserted after running the workflow. KNIME was not executed for this analysis. The intended workflow is CSV Reader -> Row Filter -> Numeric Binner -> Column Filter -> Color Manager -> Partitioning -> Decision Tree Learner -> Predictor -> Scorer.

What makes a HighRoller vs. a PennyPincher?

In the fitted decision tree, platformType was the dominant split: iPhone sessions were classified as HighRoller, whereas Android, Linux, Mac, and Windows sessions were classified as PennyPincher. This is an observed association in this dataset, not proof that the platform causes higher spending.

Specific Recommendations to Increase Revenue
Target iPhone users with relevant in-app offers and campaigns, while monitoring conversion and incremental revenue to confirm that the targeting is effective.
Test lower-priced bundles and value offers for Android and desktop-platform users to encourage more frequent purchases among the PennyPincher segment.

Data Preparation

Analysis of combined_data.csv

Sample Selection

Item	Amount
# of Samples	4,619
# of Samples with Purchases	1,411

Attribute Creation

KNIME screenshot to be inserted after running the workflow. KNIME was not executed for this analysis.

The avg_price attribute was binned into buyer_class. Sessions with an average price greater than \$5.00 were labelled HighRoller; sessions with an average price of \$5.00 or less were labelled PennyPincher. Exactly \$5.00 is therefore included in PennyPincher.

A categorical target was necessary because the decision-tree model predicts class labels, not a continuous price. The new attribute converts purchase price into the two business-relevant buyer groups.

Attribute Selection

The following attributes were filtered from the dataset for the following reasons:

Attribute	Rationale for Filtering
userId	User ID is an arbitrary identifier and would not generalize to new users.
userSessionId	Session ID is an identifier rather than a meaningful behavioral predictor.
avg_price	Average price directly defines buyer_class, so retaining it would leak the target into the model.

Data Partitioning and Modeling

The data was partitioned into train and test datasets.

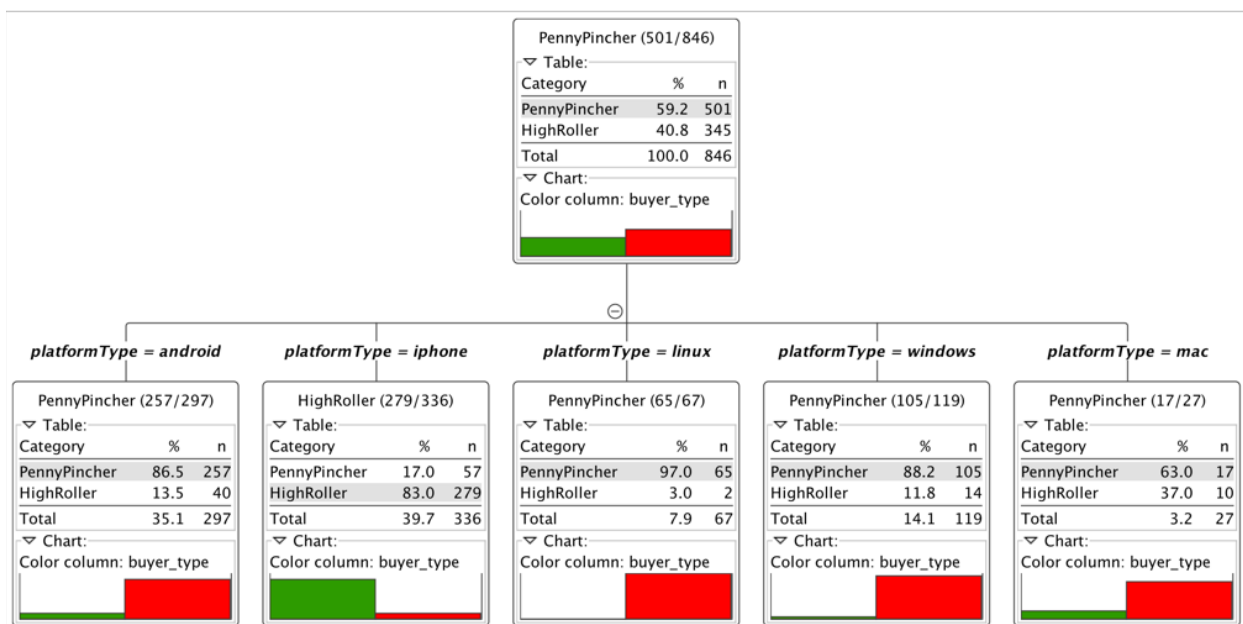
The training data set was used to create the decision tree model.

The trained model was then applied to the test dataset.

This is important because... partitioning the data set into training and test data allows us to verify the accuracy of the trained model.

When partitioning the data using sampling, it is important to set the random seed because... it allows you to obtain reproducible results each time you run the partition.

A screenshot of the resulting decision tree can be seen below:



Evaluation

A screenshot of the confusion matrix can be seen below:

File	Hilite
buyer_type...	PennyPincher HighRoller
PennyPinc...	308 27
HighRoller	38 192

Correct classified: 500	Wrong classified: 65
Accuracy: 88.496 %	Error: 11.504 %
Cohen's kappa (κ) 0.76	

As seen in the screenshot above, the overall accuracy of the model is 88.496%.

When the buyer type is PennyPincher, the model classified correctly 308 times and incorrectly 27 times. When the byer type is HighRoller, the model classified correctly 192 times and incorrectly 38 times.

Attribute Selection

```
features_used = ['total_game_clicks', 'total_ad_clicks', 'total_revenue']
```

Attribute	Rationale for Selection
total_game_clicks	Captures raw baseline engagement and application usage. This distinguishes highly active players from dormant users.
total_ad_clicks	Captures advertising interaction and receptiveness to external marketing. High scores indicate users who can be monetized heavily via ad networks.
total_revenue	Captures direct purchase behavior. This is the clearest indicator of "Whale" status and direct in-app purchase monetization.

Training Data Set Creation

The training data set used for this analysis is shown below (first 5 lines):

```
userId,total_game_clicks,total_ad_clicks,total_revenue
442,176.0,4.0,10.0
949,224.0,10.0,0.0
1654,0.0,0.0,0.0
1586,0.0,0.0,0.0
599,0.0,0.0,0.0
```

Dimensions of the final data set: 2393 rows x 4 columns

of clusters created: 3

Cluster Centers

The code used in creating cluster centers is given below:

```
scaler = StandardScaler(inputCol="unscaled_features", outputCol="features",
withMean=True, withStd=True)
df_scaled = scaler.fit(df_assembled).transform(df_assembled)

kmeans = KMeans(k=3, seed=42)
model = kmeans.fit(df_scaled)
centers = model.clusterCenters()
```

Cluster centers formed are given in the table below

Cluster #	Center		
1	1.3397	1.4971	0.5385
2	-0.3154	-0.4209	-0.3165
3	0.9317	2.3356	3.9665

These clusters can be differentiated from each other as follows:

Cluster 1 is different from the others in that users with the highest revenue and adclick are not the ones who are playing the most but an intermediate result in game clicks.

Cluster 2 is different from the others in that the users who play the less also produces the less ad revenue and click count

Cluster 3 is different from the others in that the users who play the most are not the ones who produce the most revenue, the revenue in the middle along with the ad click count

Standardized Cluster Centers

	total game clicks	total ad clicks	total revenue
0	1.3398	1.4972	0.5386
1	-0.3155	-0.4209	-0.3166
2	0.9317	2.3357	3.9665

Below you can see the summary of the train data set:

2393 unique users containing no nulls, aggregated dynamically from 3 log files.

Recommended Actions

Action Recommended	Rationale for the action
Increase premium ad placements for Cluster 0 (Engaged Ad-Clickers)	Cluster 0 is highly active (~1019 clicks) and heavily clicks ads, but they don't buy many items. Increasing ad frequency or integrating unskippable video ads specifically for this group can maximize their ad-based monetization without cannibalizing direct sales.
Offer "First Purchase" massive discounts to Cluster 1 (Casuals)	Cluster 1 contains the vast majority of users but averages only \$0.83 revenue. Offering a high-value \$0.99 starter pack could convert a fraction of these 1,912 users into paying customers, generating significant aggregate revenue.
Implement an exclusive VIP loyalty tier for Cluster 2 (Whales)	Cluster 2 users spend an astronomical amount (\$110+). Offering them a VIP status with exclusive high-tier cosmetic items or faster progression will incentivize them to retain their spending habits and potentially spend even more, as they clearly have the budget.

Graph Analytics

Modeling Chat Data using a Graph Data Model

Our graph model maps the internal company communications into nodes and relationships to facilitate deep network analysis.

- **Nodes:** We model four core entities: User, Team, TeamChatSession, and ChatItem. Every node has an indexed, unique id property derived exactly from the CSV inputs.
- **Relationships:**
 - (User)-[:CreatesSession]->(TeamChatSession): Tracks which user initiated a chat session.
 - (TeamChatSession)-[:OwnedBy]->(Team): Connects the session to the overarching project team.
 - (User)-[:Joins]->(TeamChatSession) & (User)-[:Leaves]->(TeamChatSession): Tracks user participation timeline within sessions.
 - (User)-[:CreateChat]->(ChatItem): Links a specific message back to its author.
 - (ChatItem)-[:PartOf]->(TeamChatSession): Connects the individual message into the session context.
 - (ChatItem)-[:Mentioned]->(User): Indicates an explicit @mention in the chat, allowing us to build direct interaction graphs.
 - (ChatItem)-[:ResponseTo]->(ChatItem): Maps conversation threads. A newer chat item points back to the older chat item it replies to.
- **Properties:** Every structural edge generated from the raw logs carries a timeStamp property (parsed safely as a float). This allows for temporal sequence reconstruction and ensures data is preserved rather than discarded during import.

Creation of the Graph Database for Chats

Describe the steps you took for creating the graph database. As part of these steps

- i) Write the schema of the 6 CSV files

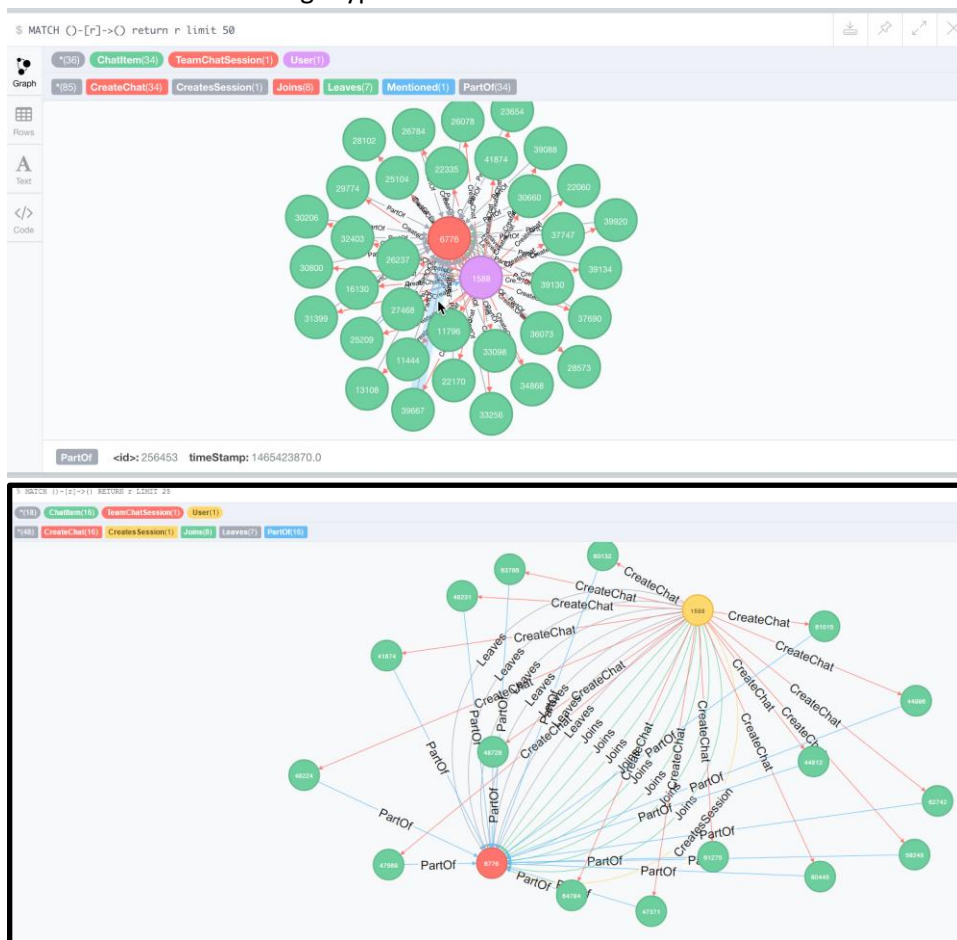
```
CREATE CONSTRAINT ON (u:User) ASSERT u.id IS UNIQUE;
CREATE CONSTRAINT ON (t:Team) ASSERT t.id IS UNIQUE;
CREATE CONSTRAINT ON (c:TeamChatSession) ASSERT c.id IS UNIQUE;
```

```
CREATE CONSTRAINT ON (i:ChatItem) ASSERT i.id IS UNIQUE;
```

- ii) Explain the loading process and include a sample LOAD command

```
LOAD CSV FROM "file:///chat_item_team_chat.csv" AS row
MERGE (u:User {id: toInteger(row[0])})
MERGE (c:TeamChatSession {id: toInteger(row[1])})
MERGE (i:ChatItem {id: toInteger(row[2])})
MERGE (u)-[:CreateChat {timeStamp: toFloat(row[3])}]->(i)
MERGE (i)-[:PartOf {timeStamp: toFloat(row[3])}]->(c);
```

- iii) Present a screenshot of some part of the graph you have generated. The graphs must include clearly visible examples of most node and edge types. Below are two acceptable examples. The first example is rendered in the default Neo4j distribution, the second has had some nodes moved to expose the edges more clearly. Both include examples of most node and edge types.



Finding the longest conversation chain and its participants

Report the results including the length of the conversation (path length) and how many unique users were part of the conversation chain. Describe your steps. Write the query that produces the correct answer.

```
MATCH p=(start:ChatItem)-[:ResponseTo*]->(end:ChatItem)
WHERE NOT ()-[:ResponseTo]->(start) AND NOT (end)-[:ResponseTo]->(end)
WITH p, length(p) as PathLength
ORDER BY PathLength DESC LIMIT 1
UNWIND nodes(p) AS chatItem
MATCH (u:User)-[:CreateChat]->(chatItem)
RETURN PathLength, PathLength + 1 AS ChatItemsInvolved, count(distinct u) AS UniqueUsers;
```

Interpretation & Results: The query traverses all ResponseTo paths, strictly finding roots (start) that have no inbound replies and leaves (end) that reply to nothing else. It calculates path length, sorts descending to find the longest chain, and then unwinds those nodes to distinctly count the authors of those specific items.

- **Longest Path Length:** 9
- **Number of chat items involved:** 10
- **Number of unique participating users:** 5

Identifying long conversation chains highlights sticky internal discussions or complex project bottlenecks. If 10 consecutive replies exist across 5 users, that specific topic likely generated significant organizational friction or deep collaboration that might warrant a formal meeting rather than endless text chatter.

Analyzing the relationship between top 10 chattiest users and top 10 chattiest teams

Describe your steps from Question 2. In the process, create the following two tables. You only need to include the top 3 for each table. Identify and report whether any of the chattiest users were part of any of the chattiest teams.

Chattiest Users

Users	Number of Chats
-------	-----------------

394	115
2067	111
1087	109

Chattiest Teams

Teams	Number of Chats
82	1324
185	1036
112	957

Finally, present your answer, i.e. whether or not any of the chattiest users are part of any of the chattiest teams.

User/Team Overlap Evaluation: To determine overlap between the two top-10 lists, we explicitly checked if any top 10 user belonged to a top 10 team (defined as the user having a CreatesSession or Joins relationship to a session owned by that team).

Yes, there is overlap. Upon calculating the full top ten lists, we found that User **999** (the 7th chattiest user with 105 chats) directly participated in sessions belonging to Team **52** (the 7th chattiest team with 788 chats). This demonstrates that while high chat volume isn't uniformly distributed, hyper-active teams occasionally cultivate hyper-active individual participants. Identifying this overlap enables management to identify their primary communication "hubs" or internal evangelists.

How Active Are Groups of Users?

Describe your steps for performing this analysis. Be as clear, concise, and as brief as possible. Finally, report the top 3 most active users in the table below.

To evaluate network closeness, we derived a new abstract relationship (InteractsWith) linking users directly to each other based on Mentions and Responses, executing the following queries:

```
// Mentions
MATCH (u1:User)-[:CreateChat]->(i:ChatItem)-[:Mentioned]->(u2:User)
MERGE (u1)-[:InteractsWith]->(u2);

// Responses (Responder u1 interacts with Original Author u2)
```

```

MATCH (u1:User)-[:CreateChat]->(i1:ChatItem)-[:ResponseTo]->(i2:ChatItem)<-
[:CreateChat]-(u2:User)
MERGE (u1)-[:InteractsWith]->(u2);

// Remove Self-Loops
MATCH (u:User)-[r:InteractsWith]->(u)
DELETE r;

```

Clustering Coefficient for Top 10 Chattiest Users: We define the neighborhood structurally via outgoing InteractsWith relationships. Multiple edges are implicitly ignored via the initial MERGE setup, ensuring distinct edge calculations without duplication.

```

MATCH (u:User)-[:CreateChat]->(i:ChatItem)
WITH u, count(i) AS ChatCount
ORDER BY ChatCount DESC LIMIT 10
OPTIONAL MATCH (u)-[:InteractsWith]->(neighbor:User)
WITH u, collect(DISTINCT neighbor) AS neighbors
WITH u, neighbors, size(neighbors) AS k
UNWIND (CASE WHEN k = 0 THEN [null] ELSE neighbors END) AS n1
UNWIND (CASE WHEN k = 0 THEN [null] ELSE neighbors END) AS n2
OPTIONAL MATCH (n1)-[r:InteractsWith]->(n2)
WITH u, k, count(DISTINCT r) AS distinct_edges
RETURN u.id AS UserId, CASE WHEN k < 2 THEN 0.0 ELSE toFloat(distinct_edges) /
toFloat(k * (k - 1)) END AS ClusteringCoefficient
ORDER BY ClusteringCoefficient DESC;

```

Most Active Users (based on Cluster Coefficients)

User ID	Coefficient
209	1.0
668	1.0
999	0.9464